

Feature Highlights

A crisp product brief for the hyperlocal daily-wage job platform - built to connect trusted workers with households needing short-term help.

The problem solved Workers no longer wait at labour chowks without certainty. Households can discover, evaluate and hire nearby help without middlemen.	Primary users Workers: painters, plumbers, cleaners, electricians, helpers and other daily-wage professionals. Households: families needing safe, short-term assistance.
Core promise Fast matching, clear job details, visible trust signals, simple actions and persistent records - designed for low-to-mid-range Android phones.	Judging focus Responsive mobile UX, consistent visual system, reusable architecture, persistent data and functional end-to-end flows.

End-to-end experience

Sign in -> choose a role -> complete profile -> discover or post jobs -> select a worker -> complete work -> settle wallet payment -> exchange feedback.

Worker journey	Household journey
Create profile, set skills and availability, browse ranked nearby opportunities, express interest, work, receive payment and build a reputation.	Create a one-day job, review interested local workers, use verification and trust indicators, hire, pay on completion and save reliable workers.

1. Identity, onboarding and profiles

Phone OTP login Mobile-number sign-in with a clearly disclosed demo OTP. Workers and households choose their role during onboarding.	Guided onboarding Captures name, language, locality, address, preferred search radius, family emergency contact and wallet PIN.
Simulated Aadhaar verification Stores only verification result and optional last four digits - never a raw Aadhaar number. Verified badge improves household confidence.	Worker skills and wage Workers define skill tags, daily or hourly expected wage, experience, biography and work-time window.
Availability controls Daily availability plus a live Available Now switch. Availability is surfaced in discovery and on the neighbourhood map.	Trust profile Public worker profile shows verification, skills, experience, completed jobs, thumbs-up/down totals, streak and earned badges.
Location and geofence Each user has locality coordinates and an adjustable radius. Distance is calculated for recommendations and nearby views.	Language preference Persistent user language setting supports the app's multilingual direction: English, Hindi, Gujarati, Spanish and French.

Privacy safeguards

Session cookies are server-only. Personal contact data is retained for approved work and safety workflows; raw Aadhaar details are intentionally excluded from persistent storage.

2. Jobs, discovery and hiring

One-day job posting Households post a title, category, description, budget, date, start time, duration and urgency. The job inherits the household location.	Job lifecycle Open -> Assigned -> In Progress -> Completed. Cancellation and notification flows keep interested workers informed.
Recommended job feed Open jobs are ranked using skill match, distance, budget fit, urgency and household trust context. Dated jobs are filtered out.	Express interest Workers can submit an interest message and optional quoted wage. Duplicate applications are blocked by the database.
Hyperlocal map Households see Available Now workers; workers see nearby open jobs. Pins display key details and support fast actions.	Applicant review Households compare applicants by quoted wage, distance, skills, verification, trust score, history and favorite status.
Worker selection A household can select one valid applicant. Other applicants are marked as not selected and receive an in-app notification.	Instant hire From the map, a household can create and assign an urgent job to an available worker in a single flow.

Favorites and repeat hiring

Households can add trusted workers to a personal shortlist. Favorite status is retained and visible when reviewing applicants again.

3. Completion, trust and wallet

Job start and completion The assigned worker can start a job. Either assigned party can mark a valid assigned/in-progress job complete.	Atomic wallet settlement Completion charges the household only when its balance is sufficient, credits the worker once, and records both sides in the wallet ledger.
Wallet PIN A four-digit PIN protects top-ups, withdrawals and wallet actions. PIN values are hashed before storage.	Wallet history Users see balance, earned and spent totals, plus a chronological transaction record with job references and notes.
Mutual feedback Both parties can leave a thumbs-up or thumbs-down, optional tags and a short note after completion. One rating per rater per job is enforced.	Trust score Worker trust combines rating signals, completed jobs, verification and reliability streak to make comparisons simple.
Streaks and badges Completed work increments streaks and unlocks milestones such as First Job, Five Jobs, Twenty Jobs, high-rating and streak badges.	In-app notifications Notifications cover welcome, job matches, interest, hiring, cancellation, start, completion, ratings and wallet events.

4. Support, architecture and quality

Support hub Built-in customer-care and chatbot experiences provide quick predefined help questions, support messages and direct assistance pathways.	Persistent PostgreSQL data Users, sessions, worker profiles, jobs, applications, ratings, favorites, wallet entries, notifications, messages and support tickets persist in PostgreSQL.
Reusable architecture Next.js pages use reusable components, domain helpers, typed database schema, server actions and a dedicated data-access layer.	Efficient data model Indexes support common feed, job, session, wallet and notification queries. Unique constraints prevent duplicate interest, ratings and favorites.
Reliability safeguards Role authorization, input validation, conditional wallet debits and transactional completion reduce inconsistent or replayed job-payment actions.	Mobile-first UX Responsive layouts, intuitive cards, prominent calls to action, readable hierarchy and consistent component styling support fast decisions on small screens.
Offline-ready direction The server-backed design keeps persistent state centralized. The architecture can be extended with queued actions and sync for weak-network conditions.	Demo readiness A local PostgreSQL setup, schema migrations and idempotent demo seeding make the full experience easy to demonstrate and evaluate.

Feature coverage at a glance

KaamSetu combines job matching, hyperlocal discovery, transparent trust, secure wallet simulation, ratings, notifications, favorites and support into a single accessible daily-wage work platform.

Prepared for KaamSetu v1.1 - product and implementation feature brief.